

Rent-City — Giải thích chi tiết 7 bảng ERD

Tài liệu thiết kế cơ sở dữ liệu · Phiên bản MVP

Dự án: Rent-City

Nhóm: 5 thành viên FPT

Phiên bản: MVP-v1 · 08/05/2026

Mục lục

- `users` — Người dùng
- `car_categories` — Loại xe
- `cars` — Xe
- `car_images` — Ảnh xe
- `bookings` — Đơn đặt xe (bảng trung tâm)
- `payments` — Thanh toán
- `reviews` — Đánh giá
- Tổng kết các quan hệ
- Q&A — Câu hỏi thường gặp khi review

1. `users` — Người dùng

9 cột

Mục đích: Lưu thông tin tất cả người dùng trong hệ thống — khách hàng, nhân viên, admin. Một bảng dùng chung cho cả 3 loại, phân biệt qua cột `role`.

Tại sao gộp 3 loại vào 1 bảng?

Vì thông tin cơ bản (email, password, tên, sdt) đều giống nhau. Tách ra 3 bảng riêng sẽ làm code đăng nhập phức tạp hơn nhiều. Đây là pattern phổ biến gọi là "single-table user authentication".

Cột	Kiểu	Giải thích
PK <code>id</code>	<code>bigint, auto_inc</code>	Khóa chính, MySQL tự sinh số tăng dần. Dùng để các bảng khác tham chiếu vào.
UK <code>email</code>	<code>varchar(100)</code>	Email đăng nhập. UNIQUE để không ai đăng ký trùng email. Dùng làm username thay vì username riêng cho gọn.
<code>password_hash</code>	<code>varchar(255)</code>	Mật khẩu đã hash bằng BCrypt. Không bao giờ lưu password gốc. BCrypt output dài ~60 ký tự, để 255 cho an toàn nếu đổi thuật toán.
<code>full_name</code>	<code>varchar(100)</code>	Tên đầy đủ, hiện trên giao diện và biên bản giao xe.
<code>phone</code>	<code>varchar(20)</code>	Số điện thoại. Để <code>varchar</code> không phải <code>int</code> vì có số 0 đầu, có thể có dấu +84.
<code>id_card_url</code>	<code>varchar(255)</code>	Đường dẫn đến file ảnh CCCD đã upload. NULL nếu chưa upload. Lưu path chứ không lưu file gốc trong DB.
<code>role</code>	<code>enum</code>	Vai trò: CUSTOMER thuê xe, STAFF làm check-in/out, ADMIN quản lý hệ thống.
<code>kyc_status</code>	<code>enum</code>	Trạng thái duyệt CCCD: PENDING (vừa upload chờ duyệt), VERIFIED (đã duyệt), REJECTED (admin từ chối). Chỉ user VERIFIED mới được đặt xe.
<code>created_at</code>	<code>timestamp</code>	Ngày tạo tài khoản. Dùng để báo cáo "user mới trong tháng".

Ví dụ thực tế trong seed data: `tj@gmail.com` có `role=CUSTOMER`, `kyc_status=VERIFIED` → được phép đặt xe. `huong.le@gmail.com` có `kyc_status=PENDING` → không được đặt xe đến khi admin duyệt.

2. car_categories — Loại xe

3 cột

Mục đích: Phân loại xe theo nhóm (Sedan, SUV, MPV, Hatchback). Đây là bảng **lookup** — bảng tra cứu chứa các giá trị cố định.

Tại sao tách ra bảng riêng mà không ghi thẳng "Sedan" vào cột của cars?

- **Tránh sai chính tả:** nếu lưu chuỗi tự do, có người gõ "Sedan", người khác gõ "sedan" → khi filter sẽ sót dữ liệu.
- **Dễ thêm/sửa:** muốn thêm loại "Pickup" chỉ cần INSERT 1 dòng, không cần sửa code.
- **Filter nhanh hơn:** query JOIN bằng id (số) nhanh hơn so sánh chuỗi.
- **Lưu thuộc tính chung:** ví dụ seats — Sedan luôn 5 chỗ, SUV 7 chỗ, không cần lặp lại ở từng xe.

Cột	Kiểu	Giải thích
PK id	bigint	Khóa chính.
UK name	varchar(50)	Tên loại xe. UNIQUE để không tạo trùng "Sedan" 2 lần.
seats	int	Số chỗ ngồi đặc trưng của loại này.

Ví dụ: Sedan-5 chỗ, SUV-7 chỗ, MPV-7 chỗ, Hatchback-5 chỗ.

3. cars — Xe

11 cột

Mục đích: Lưu thông tin từng chiếc xe cụ thể trong đội xe. Đây là **trung tâm của module Fleet**.

Lưu ý quan trọng: mỗi dòng = một chiếc xe vật lý cụ thể, không phải một mẫu xe. Công ty có 3 chiếc Toyota Vios → 3 dòng khác nhau, 3 biển số khác nhau.

Cột	Kiểu	Giải thích
PK id	bigint	Khóa chính.
FK category_id	bigint	Trỏ tới car_categories.id. Xe này thuộc loại nào (Sedan/SUV/...).
UK license_plate	varchar(20)	Biển số xe. UNIQUE vì không có 2 xe trùng biển ở Việt Nam.
brand	varchar(50)	Hãng xe (Toyota, Honda, Mazda...).
model	varchar(50)	Tên dòng xe (Vios, City, Fortuner...).
year	int	Năm sản xuất. Khách thường quan tâm xe đời nào.
transmission	enum	AUTO (số tự động) hay MANUAL (số sàn). Là filter quan trọng khi tìm xe.
price_per_day	decimal(12,0)	Giá thuê 1 ngày. Dùng decimal không dùng float vì float làm tròn sai khi tính tiền. Số 0 sau dấu phẩy vì VND không có lẻ.
deposit	decimal(12,0)	Số tiền cọc khi nhận xe. Sẽ hoàn lại nếu xe trả đúng tình trạng.
status	enum	AVAILABLE (sẵn sàng cho thuê), MAINTENANCE (đang bảo dưỡng tạm thời), RETIRED (thanh lý vĩnh viễn).
description	text	Mô tả chi tiết xe để hiện trang chi tiết. text không giới hạn độ dài như varchar.

Lưu ý về status : đây là trạng thái **dài hạn** của xe — không phải "xe có rảnh ngày X không". Việc xe có rảnh ngày cụ thể được tính từ bảng bookings .

Mục đích: Lưu danh sách ảnh của từng xe. Mỗi xe có thể có nhiều ảnh (ngoại thất, nội thất, taplo).

Tại sao tách ra bảng riêng mà không lưu trong cars?

Đây là vấn đề **chuẩn hóa 1NF** trong database. Một xe có nhiều ảnh → không thể lưu kiểu `image1`, `image2`, `image3` thành 3 cột vì:

- Nếu xe nào đó có 10 ảnh thì sao? Phải thêm cột `image10`?
- Cách đó vi phạm 1NF — mỗi cột chỉ nên chứa 1 giá trị.
- Hoặc nếu lưu kiểu chuỗi "`url1,url2,url3`" thì khi query sẽ rất khó.

→ Giải pháp: tách thành bảng con với quan hệ 1-N. Một xe có N ảnh, mỗi ảnh là 1 dòng riêng.

Cột	Kiểu	Giải thích
PK <code>id</code>	<code>bigint</code>	Khóa chính.
FK <code>car_id</code>	<code>bigint</code>	Trỏ tới <code>cars.id</code> . Ảnh này thuộc xe nào.
<code>image_url</code>	<code>varchar(255)</code>	Đường dẫn file ảnh (ví dụ <code>/uploads/cars/vios-1.jpg</code>).
<code>is_primary</code>	<code>boolean</code>	Có phải ảnh đại diện không. Mỗi xe có 1 ảnh <code>primary</code> hiện ngoài danh sách, các ảnh còn lại hiện trong gallery chi tiết.

Mẹo nghiệp vụ: khi tạo xe mới, ảnh đầu tiên upload tự động set `is_primary = true`. Khi user đổi ảnh đại diện, set ảnh cũ về `false` rồi set ảnh mới `true` (cần làm trong cùng 1 transaction).

Mục đích: Lưu mỗi đơn đặt xe của khách. Đây là **bảng quan trọng nhất** trong hệ thống — kết nối user, car, payment, review.

Cột	Kiểu	Giải thích
PK id	bigint	Khóa chính.
FK user_id	bigint	Trỏ tới users.id. Khách nào đặt.
FK car_id	bigint	Trỏ tới cars.id. Xe nào được đặt.
start_date	date	Ngày bắt đầu thuê. Dùng date không datetime vì hệ thống chỉ tính theo ngày, không theo giờ.
end_date	date	Ngày trả xe.
total_price	decimal(12,0)	Tổng tiền thuê = (end_date - start_date) × price_per_day. Lưu cố định để nếu sau này admin đổi giá xe thì booking cũ không bị thay đổi.
deposit_amount	decimal(12,0)	Số tiền cọc tại thời điểm đặt. Cũng lưu cố định lý do tương tự.
status	enum	PENDING_PAYMENT / CONFIRMED / IN_USE / COMPLETED / CANCELLED. State machine, xem ở dưới.
damage_note	text, nullable	Ghi chú hư hỏng khi trả xe. NULL nếu trả xe không có vấn đề.
damage_fee	decimal(12,0), default 0	Phí đền bù trừ vào tiền cọc. 0 nếu không có.
created_at	timestamp	Thời điểm tạo booking. Dùng để sort lịch sử và tính TTL của PENDING_PAYMENT.

State machine của status — phần này nắm rõ khi present:

```
[Khách bấm "Đặt xe"]
  ↓
PENDING_PAYMENT —(quá 15 phút chưa pay HOẶC khách hủy)→ CANCELLED
  ↓ (thanh toán thành công)
CONFIRMED —(khách hủy < 24h trước ngày nhận)→ CANCELLED
  ↓ (đến ngày nhận, NV check-in)
IN_USE
  ↓ (đến ngày trả, NV check-out, ghi damage nếu có)
COMPLETED
```

Tại sao lưu total_price thay vì tính lại mỗi lần? Đây là pattern gọi là **historical snapshot** — chụp lại giá trị tại thời điểm giao dịch. Nếu admin tăng giá xe Vios từ 700K lên 800K vào tháng sau, booking đã đặt tháng trước **không được tăng giá theo**. Lưu sẵn total_price đảm bảo điều đó.

Mục đích: Lưu thông tin thanh toán của mỗi booking.

Tại sao không gộp vào bảng bookings?

- **Tách trách nhiệm rõ ràng:** bảng booking lo nghiệp vụ thuê xe, bảng payment lo nghiệp vụ tài chính.
- **Có thể mở rộng sau:** phiên bản đầy đủ sẽ có refund (hoàn cọc), partial payment (cọc trước, phần còn lại trả sau) → cần nhiều dòng payment cho 1 booking.
- **Audit trail tốt hơn:** lịch sử thanh toán riêng biệt, dễ trace khi có sự cố.

Trong MVP này: giữ rule 1 booking = 1 payment để đơn giản, nên booking_id được set UNIQUE.

Cột	Kiểu	Giải thích
PK id	bigint	Khóa chính.
FK UK booking_id	bigint	Trỏ tới bookings.id. UNIQUE đảm bảo 1 booking chỉ có đúng 1 payment.
amount	decimal(12,0)	Số tiền thực tế thanh toán. Thường = bookings.total_price.
method	enum	MOCK (giả lập VNPAY cho project review) / CASH (khách trả tiền mặt tại quầy).
status	enum	SUCCESS / FAILED. FAILED khi mock random ra fail hoặc khách hết tiền.
paid_at	timestamp	Thời điểm thanh toán thành công. NULL nếu FAILED.

Lưu ý quan hệ 1-1: vì booking_id UNIQUE, nên dù FK là quan hệ "many" về mặt cú pháp, UNIQUE biến nó thành "one". Trong JPA Entity, dùng @OneToOne thay vì @OneToMany.

Mục đích: Lưu đánh giá của khách sau khi hoàn thành thuê xe.
Quy tắc nghiệp vụ: Chỉ khách đã thuê xe (booking COMPLETED) mới được review · Mỗi booking chỉ được review 1 lần · Review hiện trong trang chi tiết xe để khách khác tham khảo.

Cột	Kiểu	Giải thích
PK id	bigint	Khóa chính.
FK UK booking_id	bigint	Trỏ tới bookings.id. UNIQUE = mỗi booking review 1 lần.
FK user_id	bigint	Trỏ tới users.id. Ai viết review (chính là bookings.user_id).
FK car_id	bigint	Trỏ tới cars.id. Review cho xe nào (chính là bookings.car_id).
rating	int (1-5)	Số sao. Dùng CHECK constraint để giới hạn 1-5.
comment	text	Nội dung đánh giá.
created_at	timestamp	Ngày viết review.

Câu hỏi giáo viên hay hỏi: "Tại sao có cả user_id và car_id trong reviews trong khi đã có thể lấy từ booking_id?"

Trả lời chuẩn: Đây là **denormalization có chủ ý** để query nhanh hơn. Khi hiển thị reviews trên trang chi tiết xe, so sánh 2 cách:

```
-- Cách KHÔNG denormalize (chậm hơn, pha'i JOIN 2 lần)
SELECT r.*, u.full_name FROM reviews r
JOIN bookings b ON r.booking_id = b.id
JOIN users u ON b.user_id = u.id
WHERE b.car_id = ?

-- Cách CÓ denormalize (nhanh hơn, JOIN trực tiếp)
SELECT r.*, u.full_name FROM reviews r
JOIN users u ON r.user_id = u.id
WHERE r.car_id = ?
```

Lưu thêm user_id , car_id tốn ít disk nhưng tiết kiệm 1 lần JOIN — đáng vì query reviews là query rất phổ biến.

Tổng kết các quan hệ giữa 7 bảng

Để tóm gọn khi present, đây là bức tranh tổng quan của tất cả quan hệ trong hệ thống:

Quan hệ	Loại	Diễn giải bằng lời
<code>users</code> → <code>bookings</code>	1 — N	Một user có nhiều booking
<code>users</code> → <code>reviews</code>	1 — N	Một user viết nhiều review
<code>car_categories</code> → <code>cars</code>	1 — N	Một loại xe (Sedan...) có nhiều xe cụ thể
<code>cars</code> → <code>car_images</code>	1 — N	Một xe có nhiều ảnh
<code>cars</code> → <code>bookings</code>	1 — N	Một xe có nhiều booking ở các thời điểm khác nhau
<code>cars</code> → <code>reviews</code>	1 — N	Một xe nhận nhiều review
<code>bookings</code> ↔ <code>payments</code>	1 — 1	Một booking có đúng 1 payment (do UNIQUE booking_id)
<code>bookings</code> ↔ <code>reviews</code>	1 — 0..1	Một booking có tối đa 1 review (chưa review thì 0, đã review thì 1)

Q&A — Câu hỏi giáo viên hay hỏi

Câu 1: Tại sao không có bảng `roles` riêng?

Vì hệ thống chỉ có 3 vai trò cố định (CUSTOMER, STAFF, ADMIN), không cho phép user tự tạo role mới. Dùng ENUM trực tiếp đơn giản và nhanh hơn JOIN bảng. Khi nào hệ thống cần permission phức tạp (custom role, custom permission) thì mới tách bảng `roles` + `permissions`.

Câu 2: Làm sao biết xe X có rảnh ngày Y không khi không có bảng lịch riêng?

Tính từ bảng `bookings`. Xe rảnh khi không có booking nào trùng khoảng thời gian yêu cầu. Query dùng công thức `start_date < :end AND end_date > :start` để tìm khoảng chồng lấn — đây là công thức chuẩn để check overlap giữa 2 khoảng thời gian.

Câu 3: Sao bảng `bookings` không có cột `branch_id` (chi nhánh)?

Phiên bản đầy đủ có. MVP này giả định công ty chỉ có 1 chi nhánh duy nhất nên bỏ ra để đơn giản. Khi mở rộng nhiều chi nhánh, sẽ thêm bảng `branches` và `cars.branch_id` + `bookings.pickup_branch_id` / `return_branch_id`.

Câu 4: Tại sao dùng `decimal` mà không dùng `float` cho tiền?

Float là kiểu nhị phân, không biểu diễn chính xác các số thập phân (ví dụ $0.1 + 0.2 = 0.30000000000000004$). Khi cộng dồn nhiều giao dịch sẽ sai lệch. `decimal` lưu chính xác từng chữ số, là chuẩn cho mọi hệ thống tài chính. Đây là rule cứng trong nghiệp vụ ngân hàng và kế toán.

Câu 5: Khi xóa user thì các booking, review của họ có bị xóa theo không?

Không. Trong thực tế nên dùng **soft delete** — thêm cột `deleted_at` vào `users`, khi "xóa" chỉ set giá trị này thay vì xóa thật. Lý do: phải giữ lịch sử booking để báo cáo doanh thu, audit. Nếu bắt buộc cần CASCADE thật thì set `ON DELETE RESTRICT` để chặn xóa user khi còn dữ liệu liên quan.